

Сериализация данных в Python
pickle, msgpack и joblib

Что такое сериализация?

Сериализация (serialization) – это процесс преобразования структуры данных в последовательность байтов или текстовый формат, который можно сохранить в файл, передать по сети или в базу данных, а затем восстановить в исходном виде.

Десериализация (deserialization) – это процесс, обратный предыдущему, при котором ранее сериализованные данные восстанавливаются в исходную сложную структуру данных.

Зачем?

Если хочешь

- передать данные между процессами или серверами,
- сохранить данные (ML-модели, чекпоинты обучения),
- сохранить совместимости (версионированные данные),
- интегрировать разные системы,
- увеличить производительность.

`pickle`

`Pickle` - модуль Python для сериализации и десериализации объектов, входящий в стандартную библиотеку Python.

`Pickle` работает с бинарными протоколами, которые позволяют сериализовать сложные структуры данных, сохраняя связи между объектами, ссылки и даже пользовательские классы.

Всего 6 протоколов, где 0 протокол – самый старый, работающий в текстовом формате, крайне неэффективный, а 5 протокол – самый современный, быстрый, поддерживающий буферизацию больших данных.

pickle

```
import pickle
```

```
pickle.DEFAULT_PROTOCOL    # протокол по умолчанию  
pickle.HIGHEST_PROTOCOL    # самый новый доступный
```

```
pickle.dump(obj, file, protocol=None,...) # obj → file  
pickle.dumps(obj, protocol=None,...)      # obj → bytes  
pickle.load(file,...)                    # file → obj  
pickle.loads(data,...)                   # bytes → obj
```

```
pickle.Pickler(file, protocol=None,...)  
pickle.Unpickler(file,...)
```

pickle

Для пользовательских классов существуют специальные методы:

```
__getstate__ / __setstate__ # убираем/восстанавливаем объект,  
                             # который нельзя сериализовать,  
                             # из состояния
```

```
__reduce__, __reduce_ex__ # для контроля над сериализацией  
                           # и для её оптимизации
```

`pickle`

Не сериализуются:

- лямбды и вложенные функции,
- открытые файлы, сокеты, соединения с БД,
- генераторы (результат непредсказуем),
- модули, `thread locks`,
- объекты, содержащие ссылки на неподдерживаемые типы.

Главный недостаток `pickle` – небезопасность:
`pickle.load()` никак не проверяет объект, который десериализует, поэтому может запустить вредоносный код.

Никогда не загружайте `pickle`-файлы из неизвестных источников, используйте альтернативные форматы (`JSON`, `MsgPack`, `YAML`), если данные приходят от пользователя.

pickle

```
import pickle

data = {"user": "Alice", "scores": [1, 2, 3]}

# Объект → байты
blob = pickle.dumps(data)

# Байты → объект
restored = pickle.loads(blob)

# Сохранение состояния класса
with open("state.pkl", "wb") as f:
    pickle.dump(model, f)

with open("state.pkl", "rb") as f:
    model = pickle.load(f) # Произвольный код
```

msgpack

MessagePack (msgpack) — это эффективный формат бинарной сериализации данных.

```
pip install msgpack
```

```
import msgpack
```

<code>msgpack.packb(obj)</code>	<code># obj → bytes</code>
<code>msgpack.unpackb(packed)</code>	<code># bytes → obj</code>
<code>msgpack.pack(obj, file_object)</code>	<code># obj → file</code>
<code>msgpack.unpack(file_object)</code>	<code># file → obj</code>

msgpack

msgpack, в отличие от pickle, по умолчанию является эффективным форматом для межъязыкового обмена данными (Python → MessagePack → C++, Java → MessagePack → Python).

Поддерживает только базовые типы (dict, list, str, int, float, bool, None, bytes).

Так как не выполняет код, является безопасным.

Хорошо подходит для:

- API,
- микросервисов,
- сетевых приложений,
- межпроцессного обмена,
- распределённых систем.

msgpack

```
import msgpack

data = {"service": "auth", "cpu": 0.42, "requests": 128}

# Объект → байты (компактнее JSON)
packed = msgpack.packb(data, use_bin_type=True)
data = msgpack.unpackb(packed, raw=False)

# Поточковая запись логов
with open("events.msgpack", "wb") as f:
    for event in events:
        msgpack.pack(event, f, use_bin_type=True)

# Чтение поэлементно (без загрузки всего файла)
with open("events.msgpack", "rb") as f:
    for event in msgpack.Unpacker(f, raw=False):
        process(event)
```

joblib

Joblib – библиотека Python, предназначенная для эффективной работы с сериализацией, кэшированием, параллельными вычислениями, большими массивами данных. Оптимизирована для больших данных и NumPy-массивов.

pip install joblib

```
import joblib
```

```
# obj → file
```

```
joblib.dump(obj, file, compress=0, protocol=None)
```

```
# file → obj
```

```
joblib.load(file, mmap_mode=None)
```

joblib

Одна из интересных возможностей Joblib - memory mapping (`mmap_mode`): большие данные не обязательно полностью копировать в оперативную память при загрузке.

Так же присутствует возможность сжимать объекты при сериализации (`compress`).

`joblib.load()` использует `pickle` под капотом, поэтому имеет те же уязвимости безопасности.

joblib идеально подходит для больших NumPy-вычислений, ML, обработки изображений, научных вычислений.

joblib

Одна из интересных возможностей Joblib - memory mapping (`mmap_mode`): большие данные не обязательно полностью копировать в оперативную память при загрузке.

Так же присутствует возможность сжимать объекты при сериализации (`compress`).

`joblib.load()` использует `pickle` под капотом, поэтому имеет те же уязвимости безопасности.

joblib идеально подходит для больших NumPy-вычислений, ML, обработки изображений, научных вычислений.

joblib

```
import joblib
import numpy as np
from joblib import Memory

# Сохранение модели sklearn
joblib.dump(model, "model.joblib", compress=3)
model = joblib.load("model.joblib")

# Большой массив – ленивая загрузка через mmap
arr = joblib.load("big.joblib", mmap_mode="r")
print(arr[0, :5]) # читается только нужное, RAM не тратится

memory = Memory(location="./cache")

@memory.cache
def heavy(n):
    ...
```

Проблема совместимости версий

Сериализация не всегда означает, что файл будет вечным (Python 3.X → `serialize` → `file.pkl` → Python 4.X → ???). Совместимость зависит от формата, версии протокола и самих классов.

Поэтому для долгосрочного хранения данных часто лучше использовать более независимые форматы (JSON, CSV, другие стандартизированные форматы).

Итог

Сериализация является фундаментальным механизмом современной разработки программного обеспечения.

Она позволяет преобразовывать объекты в представление, пригодное для: хранения, передачи, кэширования, межпроцессного взаимодействия, восстановления состояния программы.

Средства сериализации в Python:

- `pickle` – для Python-объектов.
- `MessagePack` – для эффективного обмена структурированными данными.
- `Joblib` – для больших Python/NumPy-объектов и задач машинного обучения.

Интересные источники

- Документация `pickle`
- Документация `MessagePack`
- Документация `Joblib`

Что не вошло, но очень полезно:

- `Joblib` для параллельных вычислений
- Оптимизации работы Jupyter notebook при помощи параллельных вычислений (Библиотека `Joblib`)